Transport Layer

TCP/IP

2026-04-16

Table of contents

1 Transport Layer **3**
- 1.1 . . . 3
- 1.2 Internet Transport Protocol . . . 3

2 Multiplexing & Demultiplexing **3**
- 2.1 . . . 3
- 2.2 UDP (Connectionless) Demux . . . 4
- 2.3 TCP (Connection-oriented) Demux . . . 4

3 UDP (User Datagram Protocol) **4**
- 3.1 Segment . . . 4
- 3.2 UDP . . . 4
- 3.3 UDP Checksum . . . 5
  - 3.3.1 . . . 5
  - 3.3.2 1 (One's Complement) . . . 5
  - 3.3.3 (16-bit) . . . 5
  - 3.3.4 . . . 6

4 Principles of Reliable Data Transfer (rdt) **6**
- 4.1 . . . 6
- 4.2 rdt 2.0 — . . . 6
- 4.3 rdt 3.0 — Stop-and-Wait . . . 6
  - 4.3.1 rdt 3.0 4 . . . 7

5 Pipelined Protocols **7**
- 5.1 . . . 7
- 5.2 Go-Back-N (GBN) vs Selective Repeat (SR) . . . 7

6 Window Size — **8**
- 6.1 . . . 8
- 6.2 GBN . . . 8
- 6.3 SR . . . 8

6.4 SR Dilemma — N/2 ? . . . 8
6.4.1 Seq# . . . 8
6.4.2 . . . 9

7 TCP **9**
7.1 Segment . . . 9
7.2 TCP Sequence Number & ACK . . . 10
7.2.1 TCP "byte stream" . . . 10
7.2.2 Telnet (p.54) . . . 10
7.3 RTT . . . 11
7.3.1 EWMA (Exponentially Weighted Moving Average) . . . 11
7.3.2 (DevRTT) . . . 11
7.3.3 Timeout Interval . . . 11
7.3.4 Retransmission RTT ? . . . 11
7.4 TCP Reliable Data Transfer . . . 12
7.4.1 . . . 12
7.4.2 ACK (RFC 1122, 2581) . . . 12
7.4.3 Cumulative ACK (p.62) . . . 12
7.4.4 Fast Retransmit . . . 12
7.5 TCP Flow Control . . . 13
7.5.1 . . . 13
7.5.2 . . . 13
7.5.3 rwnd = 0 . . . 13
7.5.4 Flow Control vs Congestion Control . . . 13
7.6 TCP Connection Management . . . 14
7.6.1 3-way Handshake . . . 14
7.6.2 Connection Teardown . . . 14

8 Principles of Congestion Control **14**
8.1 (Congestion) ? . . . 14
8.2 (3) . . . 14
8.2.1 Scenario 2 : Goodput vs Throughput . . . 14
8.2.2 Scenario 3: Upstream . . . 15

9 TCP Congestion Control **15**
9.1 AIMD (Additive Increase Multiplicative Decrease) . . . 15
9.2 Slow Start . . . 15
9.3 TCP Tahoe vs TCP Reno . . . 16
9.3.1 . . . 16
9.3.2 TCP Tahoe (1988) . . . 16
9.3.3 TCP Reno (1990) . . . 16
9.3.4 cwnd . . . 17
9.3.5 TCP . . . 17
9.4 TCP Throughput . . . 17
9.5 TCP Fairness . . . 18
9.6 ECN (Explicit Congestion Notification) . . . 18

10 **18**

1 Transport Layer

1.1

Transport layer **application process** (logical communication) .

Network layer host host
Transport layer **process** **process**

- : App segment → Network layer
- : Segment → App layer

: Household Analogy

Ann 12 Bill 12 :

- Houses = **hosts**
- Kids = **processes**
- Letters = **app messages**
- Ann/Bill ( ) = **transport protocol (demux)**
- = **network-layer protocol**

1.2 Internet Transport Protocol

	TCP	**UDP**
	,	,
	Connection-oriented	Connectionless
	Flow control	
	Congestion control	
Header	20 bytes	8 bytes
	HTTP, FTP, SMTP	DNS, streaming, SNMP

---

2 Multiplexing & Demultiplexing

2.1

- **Multiplexing ( )**: transport header network layer
- **Demultiplexing ( )**: header segment

2.2 UDP (Connectionless) Demux

- **2-tuple**: `(dest IP, dest port)`
- (src IP, src port) **dest port →**

2.3 TCP (Connection-oriented) Demux

- **4-tuple**: `(src IP, src port, dest IP, dest port)`
- dest port 80 src demux



TCP 4-tuple , (80) .

---

3 UDP (User Datagram Protocol)

3.1 Segment

```
source port #      dest port #       16 + 16 bits

    length          checksum         16 + 16 bits

       application data
```

- **length**: UDP segment (header + data)

3.2 UDP

- No connection establishment →
- No connection state →
- Small header (8 bytes vs TCP 20 bytes)
- No congestion control →

3.3 UDP Checksum

3.3.1

"segment " , **Pseudo Header + UDP Header + UDP Data** .

Pseudo Header (IP header):

```
        Source IP Address (32 bits)

     Destination IP Address (32 bits)

  zero (8b)      Protocol=17 (8b)

           UDP Length (16 bits)
```

> Pseudo Header ?
>
> UDP header IP . Pseudo header **IP** **(wrong delivery)** .

3.3.2 1 (One's Complement)

1 :

```
0110 1010  →  1001 0101  (1  )
```

: $x+ \sim x = \underbrace{1111 \ldots 1}_{\text{all ones}}$

3.3.3 (16-bit)

```
  1111 0011 0011 0011 0         ←      (17 ,     carry)
+ 1110 1010 1010 1010 1

  1101 1101 1101 1101 1         ← carry out
                  + 1          ← wrap around (carry      )

  1101 1101 1101 1110 0         ←   sum

Checksum = 1   :
  0010 0010 0010 0001 1         ← checksum
```

3.3.4

: **16-bit word + checksum**

$$\text{sum} + \text{checksum} = \underbrace{1111 \ldots 1}_{\text{all ones}} \Rightarrow$$

$$\text{sum} + \text{checksum} \neq \underbrace{1111 \ldots 1}_{\text{all ones}} \Rightarrow$$

⚠

. (best-effort)

---

4 Principles of Reliable Data Transfer (rdt)

4.1

rdt 1.0	-	(trivial)
rdt 2.0	checksum, ACK/NAK	
rdt 2.1	Sequence number (0,1)	ACK/NAK
rdt 2.2	NAK , duplicate ACK	NAK-free
rdt 3.0	Countdown timer	

4.2 rdt 2.0 —

ACK/NAK sender .

- → duplicate
- → **Sequence number** (rdt 2.1)

4.3 rdt 3.0 — Stop-and-Wait

$$U_{sender} = \frac{L/R}{RTT + L/R}$$

: 1 Gbps link, RTT = 30ms, L = 8000 bits

$$U_{sender} = \frac{0.008}{30.008} \approx 0.00027 \quad (0.027\%!)$$

→

4.3.1 rdt 3.0 4

(a)	-	
(b) Packet loss		timeout →
(c) ACK loss	ACK	timeout → , receiver duplicate re-ACK
(d) Premature timeout	ACK	seq# duplicate

---

5 Pipelined Protocols

5.1

3 :

$$U_{sender} = \frac{3 \cdot L/R}{RTT + L/R}$$

→ 3

5.2 Go-Back-N (GBN) vs Selective Repeat (SR)

	GBN	**SR**
Sender window	N in-flight	N in-flight
Receiver window	**1**	**N**
ACK	Cumulative ACK	ACK
Out-of-order	**Discard** ( ) window unacked pkt	**Buffer** **pkt**
Timer	oldest unacked pkt	pkt

---

6 Window Size —

6.1

window size :

$$\boxed{W_{sender} + W_{receiver} \leq \text{Seq\# space (N)}}$$

Sender seq# Receiver " " seq# .

6.2 GBN

GBN receiver window = **1** :

$$W + 1 \leq N \implies W \leq N - 1 = 2^k - 1$$

- k: seq# , $N = 2^k$: seq#

6.3 SR

SR receiver window = sender window = **W** :

$$W + W \leq N \implies W \leq \frac{N}{2}$$

!

GBN $(W \leq 2^k - 1)$ SR $(W \leq 2^k/2)$. GBN receiver window 1 .
( ) , receiver window sender window .

6.4 SR Dilemma — N/2 ?

6.4.1 Seq#

Seq# (circular) . (N=4: 0,1,2,3)

W = 3 (> N/2 = 2) →

```
Step 1: Sender [0,1,2] → pkt 0,1,2
Step 2: ACK 0,1,2
        Receiver window  : [3,0,1]   ←
Step 3: Sender timeout → pkt 0
```

```
 :    seq# 0  receiver window [3,0,1]   !
      Receiver: "       "
```

```
      0  ←
    /   \
  3       1
    \   /
      2

Receiver window = [3, 0, 1]  ← 0    ( !)
Sender window   = [0, 1, 2]
```

W = 2 (= N/2) →

```
Step 1: Sender [0,1] → pkt 0,1
Step 2: ACK 0,1
        Receiver window  : [2,3]   ←
Step 3: Sender timeout → pkt 0

 :    seq# 0  receiver window [2,3]  !
      Receiver: "old data →  "


      0  ←
    /   \
  3       1
    \   /
      2

Receiver window = [2, 3]  ← 0    (   !)
Sender window   = [0, 1]
```

6.4.2

window N .

$2W > N \Rightarrow$ $>$ $\Rightarrow$

$2W \leq N \Rightarrow$ non-overlapping

---

7 TCP

7.1 Segment

```
   source port         dest port        16 + 16 bits
```

```
           sequence number              32 bits

         acknowledgement number         32 bits

 hlen   flags        receive window

  checksum      urg data pointer

           options (variable)

         application data
```

: URG, **ACK**, PSH, RST, **SYN**, **FIN**

7.2 TCP Sequence Number & ACK

7.2.1 TCP "byte stream"

TCP .

- **Seq#**: segment
- **ACK#**: (= + 1)
- **Cumulative ACK**: " "

7.2.2 Telnet (p.54)

: A seq# = 42, B seq# = 79

```
A  [Seq=42, ACK=79, data='C']       B

     Seq=42:  "         42 "
     ACK=79:  "B  79       "
     data='C': 1

A   [Seq=79, ACK=43, data='C']     B

     Seq=79:  "         79 "
     ACK=43:  "A  42 (1  )
               →   43    "

A  [Seq=43, ACK=80]            B

     ACK=80:  "B  79 (1  )
               →   80    "
```

ACK = Seq + data

ACK# “X ” = “X-1 ” . Seq=42, 1 → ACK=43 (42+1).

7.3 RTT

7.3.1 EWMA (Exponentially Weighted Moving Average)

$$\text{EstimatedRTT} = (1 - \alpha) \cdot \text{EstimatedRTT} + \alpha \cdot \text{SampleRTT}$$

$$\alpha = 0.125 \text{ (typical)}$$

→ .

7.3.2 (DevRTT)

$$\text{DevRTT} = (1 - \beta) \cdot \text{DevRTT} + \beta \cdot |\text{SampleRTT} - \text{EstimatedRTT}|$$

$$\beta = 0.25 \text{ (typical)}$$

7.3.3 Timeout Interval

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}$$

$\text{mean} + 4\sigma$, **99.99%** . safety margin .

7.3.4 Retransmission RTT ?

Ambiguity (Karn's Algorithm):

```
t=0:  pkt   (seq=100)
t=5:  timeout →
t=8:  ACK(100)

SampleRTT = ?
     : 8 - 0 = 8ms  (       )
      : 8 - 5 = 3ms  (        )
  →       ACK      !
```

RTT : - → Timeout → - → Timeout →

7.4 TCP Reliable Data Transfer

7.4.1

1. **Timeout**: unacked segment
2. **3x Duplicate ACK**: Fast Retransmit

7.4.2 ACK (RFC 1122, 2581)

segment, pending ACK	**500ms** ACK (Delayed ACK)
segment, pending ACK	cumulative ACK
Out-of-order segment (Gap)	duplicate ACK
Gap segment	ACK

7.4.3 Cumulative ACK (p.62)

```
Host A                                  Host B

Seq=92  (8 bytes)
Seq=100 (20 bytes)              ← ACK=100
                                       (B  92, 100      buffer   )
timeout  !
Seq=92

                                    ACK=120
```

ACK=120 ? B Seq=92(8 bytes) Seq=100(20 bytes) buffer : - 92~99 (8 bytes) + 100~119 (20 bytes) = **119** - → ACK=120: "120 (119)"

Cumulative ACK : ACK=100 , ACK=120 100~119 .

7.4.4 Fast Retransmit

3 duplicate ACK → timeout

```
A  [Seq=92]                  B
A  [Seq=100]          X ( )
A  [Seq=120]                 B
A  [Seq=140]                 B

B             ACK=100 (dup)
B             ACK=100 (dup)
B             ACK=100 (dup)  ← 3 !
```

```
A: timeout      Seq=100
```

7.5 TCP Flow Control

7.5.1

buffer overflow (cf. Congestion control)

7.5.2

$$\text{rwnd} = \text{RcvBuffer} - [\text{LastByteRcvd} - \text{LastByteRead}]$$

- **RcvBuffer**: (4096 bytes)
- **LastByteRcvd**:
- **LastByteRead**: Application

Receiver TCP segment header **rwnd** .

Sender :

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{rwnd}$$

7.5.3 rwnd = 0

Receiver rwnd=0 sender . buffer sender (trigger).

: sender **1 byte probe segment** . Receiver ACK rwnd .

7.5.4 Flow Control vs Congestion Control

Flow Control	Congestion Control
buffer	
rwnd (receiver → sender)	packet loss, delay
rwnd sender	cwnd sender
`min(rwnd, cwnd)`	

7.6 TCP Connection Management

7.6.1 3-way Handshake

```
Client                                Server
    [SYN, Seq=x]
    [SYNACK, Seq=y, ACK=x+1]
    [ACK, ACK=y+1]

Client state: CLOSED → SYN_SENT → ESTAB
Server state: LISTEN → SYN_RCVD → ESTAB
```

2-way handshake : , , half-open connection .

7.6.2 Connection Teardown

```
Client              Server
    [FIN]          Client: FIN_WAIT_1 → FIN_WAIT_2
    [ACK]          Server: CLOSE_WAIT
    [FIN]          Server: LAST_ACK
    [ACK]          Client: TIMED_WAIT (2×MSL  ) → CLOSED
                          Server: CLOSED
```

TIMED_WAIT: 2 × MSL(Maximum Segment Lifetime) . duplicate segment .

---

8 Principles of Congestion Control

8.1 (Congestion) ?

“ source ”

Flow control : - Flow control: **buffer** - Congestion control:

8.2 (3)

8.2.1 Scenario 2 : Goodput vs Throughput

Perfect knowledge ( ):

```
lin' = R/2     :
  =      + (          )

         drop  =
            →            R/2

  goodput   R/2   (asymptotic)
```

> p.83
>
> "sending at R/2 goodput R/2 ?"
> ( ) . R/2 .
> **premature timeout** bandwidth goodput < R/2 .

Realistic (timeout):

$$\text{goodput} < \frac{R}{2}$$

8.2.2 Scenario 3: Upstream

drop , **capacity** .

---

9 TCP Congestion Control

9.1 AIMD (Additive Increase Multiplicative Decrease)

- **Additive Increase**: loss RTT cwnd += 1 MSS
- **Multiplicative Decrease**: loss cwnd = cwnd / 2

$$\text{sending rate} \approx \frac{\text{cwnd}}{\text{RTT}} \text{ bytes/sec}$$

9.2 Slow Start

- `cwnd = 1 MSS`
- RTT **2** (ACK +1 MSS →)
- `ssthresh` linear(CA)

cwnd

← ssthresh

(linear)

( )

time

(Slow Start)

9.3 TCP Tahoe vs TCP Reno

9.3.1

loss ?

Timeout	3 Dup ACK
	, 3 →

9.3.2 TCP Tahoe (1988)

loss :

```
Timeout   : cwnd = 1 MSS, ssthresh = cwnd_before/2, Slow Start
3 Dup ACK  : cwnd = 1 MSS, ssthresh = cwnd_before/2, Slow Start
```

9.3.3 TCP Reno (1990)

3 Dup ACK **Fast Recovery** :

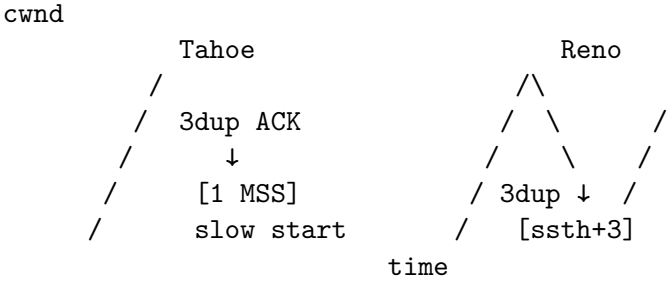
```
Timeout  :
  → cwnd = 1 MSS
  → ssthresh = cwnd / 2
  → Slow Start     (Tahoe   )

3 Dup ACK   (Fast Recovery):
  → ssthresh = cwnd / 2
  → cwnd = ssthresh + 3 MSS  ← 3  dup ACK = 3  pkt
  →   segment
  →   dup ACK : cwnd += 1 MSS  (window inflation)
  → New ACK  : cwnd = ssthresh, CA
```

Window Inflation

Fast Recovery dup ACK cwnd += 1 MSS :
dup ACK **1** . sender 1 .

9.3.4 cwnd



Tahoe: loss → (1 MSS)
Reno: 3dup → ssthresh ,

9.3.5 TCP

Tahoe	1988	congestion control, loss = cwnd=1
Reno	1990	Fast Recovery (3 dup ACK)
NewReno	1999	Reno partial ACK
CUBIC	2008	Linux , cubic cwnd
BBR	2016	bandwidth & RTT (Google)

9.4 TCP Throughput

Window size W :

$$\text{avg throughput} = \frac{3}{4} \cdot \frac{W}{\text{RTT}}$$

Loss L (Mathis 1997):

$$\text{TCP throughput} = \frac{1.22 \cdot \text{MSS}}{\text{RTT} \cdot \sqrt{L}}$$

: MSS=1500B, RTT=100ms, 10 Gbps:

$$L \leq 2 \times 10^{-10} \quad (\quad \text{loss rate} \quad)$$

9.5 TCP Fairness

K TCP bandwidth R → R/K

AIMD : throughput "equal bandwidth" zigzag (additive increase = 1 , multiplicative decrease =)

9.6 ECN (Explicit Congestion Notification)

- IP ToS field 2
- TCP ACK **ECE bit** →
-

10

! TCP

TCP Tahoe/Reno **loss = congestion** .
: - → (!) - → - **(fading)** → loss
TCP congestion → cwnd →
BBR, CUBIC, TCP (: TCP-Westwood, TCP-Veno) .

11

$U_{sender} = \frac{L/R}{RTT+L/R}$	Stop-and-wait sender utilization
$W_{GBN} \leq 2^k - 1$	GBN window size
$W_{SR} \leq \frac{2^k}{2}$	SR window size
$\text{EstRTT} = (1-\alpha)\text{EstRTT} + \alpha \cdot \text{SampleRTT}$	EWMA RTT ($\alpha = 0.125$)
DevRTT = $(1-\beta)\text{DevRTT} + \beta \cdot \lVert\text{SampleRTT} - \text{EstRTT}\rVert$	RTT ($\beta = 0.25$)
$\text{TimeoutInterval} = \text{EstRTT} + 4 \cdot \text{DevRTT}$	TCP timeout interval
rwnd = RcvBuffer − [LastByteRcvd − LastByteRead]	Flow control
avg throughput = $\frac{3W}{4 \cdot RTT}$	TCP
TCP throughput = $\frac{1.22 \cdot \text{MSS}}{\text{RTT}\sqrt{L}}$	Loss (Mathis)